

MODULE 06 · PART 2 - WINDOW FUNCTIONS

Looking Backwards and Forwards

LAG, LEAD and the value functions. Reading another row rather than summarising a set of them.

PART 2 of 8 MODULE 6 of 26 RUNS about 34 min

These functions do not summarise anything. They **fetch a value from another row.**

Simpler, and not easier

- ◆ **There is no arithmetic to get wrong.** No accumulation, no denominator, no frame on half the family.
- ◆ **But every one of them has an edge with no row in it.** The first row, the last row, the row that was null.
- ◆ **And SQL's answer to 'no row' is always null,** which then propagates through whatever you built on top.
- ◆ **So the work is at the edges, not in the middle.** That is a different kind of care from the last two modules.

THE TAKEAWAY

Aggregates fail in the middle of a partition. Navigation functions fail at the ends of one, and the ends are where every partition has exactly one of each.

The family, in one table

FUNCTION	WHICH ROW IT READS
<code>LAG(x, n)</code>	n rows earlier in the ordering. Default n is 1.
<code>LEAD(x, n)</code>	n rows later. A negative offset makes it a LAG.
<code>FIRST_VALUE(x)</code>	The first row of the frame.
<code>LAST_VALUE(x)</code>	The last row of the frame, which by default is this one.
<code>NTH_VALUE(x, n)</code>	The nth row of the frame, counting from the start.

THE TAKEAWAY

The first two are positional and take no frame. The last three are frame-based and take no offset. That split explains every difference in their behaviour.

Two halves, two sets of rules

- ◆ **LAG and LEAD cannot take a frame at all.** The Redshift documentation says so outright, and the standard agrees.
- ◆ **So they are immune to module five.** No default frame, no RANGE, no peers. A relief, and worth knowing.
- ◆ **FIRST_VALUE, LAST_VALUE and NTH_VALUE are frame-sensitive**, so everything from module five applies to them in full.
- ◆ **All five depend completely on the ordering**, which means module two applies to every one of them.

THE TAKEAWAY

One family, two rule sets. Half of these functions carry module five's baggage and half do not, and nothing in the syntax tells you which is which.

What people build with these

- ◆ **Period over period.** Day on day, week on week, year on year. The single most requested measure in analytics.
- ◆ **Elapsed time between events.** Time to next purchase, time since last login, session boundaries.
- ◆ **Change detection.** When did this value last move, and what did it move from.
- ◆ **Carry forward.** The last known price, reading, status or balance, projected across days that have none.

THE TAKEAWAY

Four shapes, and the last two modules of Part 4 are built on them. Sessionisation is elapsed time and gaps-and-islands is change detection.

Period over period, without a self-join. This is the reason **most people** learn window functions.

Day over day, in one line

```
1 SELECT store_key, sale_date, daily_sales,  
2       LAG(daily_sales) OVER (  
3         PARTITION BY store_key  
4         ORDER BY sale_date  
5       ) AS prev_day_sales,  
6       daily_sales - LAG(daily_sales) OVER (  
7         PARTITION BY store_key  
8         ORDER BY sale_date  
9       ) AS day_change  
10 FROM agg_store_day;
```

- 3 The partition is what stops a store reading the previous store's last day. Leave it out and 412 rows a day are wrong.
- 9 The first day of each store returns null, and null minus anything is null. That is correct and it still needs handling.

THE TAKEAWAY

Two things every LAG needs and half of them have: a PARTITION BY that matches the series, and a decision about the first row.

The three arguments

- ◆ **The expression**, which is any expression, not only a column. LAG of a CASE is a useful move and it comes up later.
- ◆ **The offset**, defaulting to 1. An offset of 0 returns the current row, which is legal and occasionally handy.
- ◆ **The default value**, returned when the offset falls outside the partition. This is the argument nobody uses.
- ◆ **Redshift's LAG takes no default argument.** Postgres, Snowflake and SQL Server do. COALESCE is the portable answer.

THE TAKEAWAY

The third argument turns the first row of every partition from a null into a decision. Writing it down is the difference between handling the edge and discovering it.

What it replaced

THE SELF-JOIN

Fragile at the edges

```
1 SELECT a.sale_date, a.daily_sales,  
2        b.daily_sales AS prev  
3 FROM agg_store_day a  
4 LEFT JOIN agg_store_day b  
5   ON b.store_key = a.store_key  
6   AND b.sale_date = a.sale_date - 1
```

Assumes yesterday exists as a row. On a closed day there is no match, and the chain silently breaks.

THE WINDOW

Positional, and safe

```
1 SELECT sale_date, daily_sales,  
2        LAG(daily_sales) OVER (  
3          PARTITION BY store_key  
4          ORDER BY sale_date  
5        ) AS prev  
6 FROM agg_store_day
```

Reads the previous row that exists. On a closed day it reaches back to the last trading day instead.

THE TIEBREAKER

Note they are not equivalent. One asks for yesterday and the other asks for the previous row, and those differ by exactly the gaps in the series.

LEAD, and the symmetry

```
1 SELECT member_durable_key, event_ts, event_type,  
2         LAG(event_ts) OVER w AS prev_event,  
3         LEAD(event_ts) OVER w AS next_event,  
4         LEAD(event_ts) OVER w - event_ts AS time_to_next  
5 FROM fact_app_event  
6 WINDOW w AS (PARTITION BY member_durable_key ORDER BY event_ts);
```

3 The last event of each member returns null for next_event, exactly as the first returns null for prev_event.

THE TAKEAWAY

Both functions on one window definition. The engine sorts once and reads a small buffer in both directions.

LEAD is LAG with the sign flipped

- ◆ **LEAD(x, n) is exactly LAG(x, -n)** where the engine accepts a negative offset, and Snowflake documents this explicitly.
- ◆ **Prefer the one that reads correctly.** Next is LEAD and previous is LAG, and nobody should have to work out a sign.
- ◆ **Reversing the ORDER BY is the third way**, and it changes the meaning of every other window in the same clause.
- ◆ **Which is why the named WINDOW clause matters.** Reverse a shared definition and you move four columns, not one.

THE TAKEAWAY

Three ways to look backwards and one of them is readable. There is no performance difference, so choose on behalf of whoever reads it next.

Offsets other than one

```
1 SELECT sale_date, daily_sales,  
2         LAG(daily_sales, 1) OVER w AS yesterday,  
3         LAG(daily_sales, 7) OVER w AS same_day_last_week,  
4         LAG(daily_sales, 0) OVER w AS today_again,  
5         LAG(daily_sales, 7, 0) OVER w AS last_week_or_zero  
6 FROM agg_store_day  
7 WINDOW w AS (PARTITION BY store_key ORDER BY sale_date);
```

- 3 Seven rows back, which is same day last week only if every day has a row.
- 5 The third argument. Zero instead of null for the first seven rows of each store.

THE TAKEAWAY

An offset of seven is the most common non-default offset in retail, and it is correct exactly when the series is complete.

Offsets are rows, and rows are not days

- ◆ **LAG(x, 7) means seven rows back**, which is seven days only on a complete daily series with no gaps and no duplicates.
- ◆ **Two rows on one day makes it six days**, and a closed day makes it eight. Neither is visible in the output.
- ◆ **Join to dim_date first**, so that every day has exactly one row and the offset means what it says.
- ◆ **Or compare against the date directly**, with a self-join on sale_date minus seven, which is slower and says what it means.

THE TAKEAWAY

A fixed offset is a promise about the shape of your data. Completing the series against a date dimension is how you make that promise true.

Offset, or date arithmetic

LAG WITH AN OFFSET

Positional

- One expression, no join
- Cheap once sorted
- Assumes a complete series
- Silently wrong across a gap

JOIN ON A DATE

Calendrical

- Asks for a specific date
- Costs a join
- Needs no assumption
- Returns null across a gap

THE TIEBREAKER

The window is wrong across a gap and the join is null across one. Null is the better failure, because somebody notices a null.

LAG gives you the **previous row**. It does not give you **yesterday**.

Previous row, or previous day

```
1 -- previous row that exists
2 LAG(daily_sales) OVER (PARTITION BY store_key
3                        ORDER BY sale_date)
4
5 -- previous day, whether it traded or not
6 LAG(daily_sales) OVER (PARTITION BY store_key
7                        ORDER BY sale_date)
8 -- over a series completed against dim_date,
9 -- so closed days exist as rows with zero or null
```

6 The expression is identical. The difference is entirely in what rows were in the table before the window ran.

THE TAKEAWAY

The fix is never in the LAG. It is in completing the series first, against dim_date, so that the previous row and the previous day are the same row again.

A day-over-day sales change report is built with LAG over `daily_sales`, ordered by `store_key` and then `sale_date`. The query has no PARTITION BY. It produces one row per store per day across 412 stores.

Analysts notice that a small number of rows show enormous swings, sometimes several hundred percent. The affected rows change when the report is rerun for a different date range. The totals across the report are correct.

What is wrong?

- A The ORDER BY should be `sale_date` then `store_key`, so that the same day is compared across stores.
- B There is no PARTITION BY, so the first row of each store reads the last row of the store before it.
- C LAG returns null on the first row, and something downstream is coalescing that null to zero.
- D LEAD with a negative offset should be used instead, since it handles partition boundaries correctly.

◆ PAUSE AND DECIDE

What is wrong?

A Reorder the ORDER BY

Makes it worse. Every row would then read a different store's figure, so all 412 stores would be wrong rather than one row each.

C A null coalesced downstream

Would show as a swing of exactly 100%, uniformly, on the true first row. The observed swings vary, which rules it out.

B No PARTITION BY

The sort keeps stores together, so each store's first row reads the previous store's last day. 411 wrong rows per run, spread through the report.

D Use LEAD instead

LEAD has the same boundary behaviour in the other direction. Neither function knows about partitions unless you write one.

THE SIGNAL

"The affected rows change when the date range changes." The boundary moves because the first row of each store moves. A real data defect would stay on the same rows.

FIRST_VALUE, LAST_VALUE and NTH_VALUE read **from the frame**, so module five has not gone away.

All three, with the frame stated

```
1 SELECT product_key, effective_from, price,  
2         FIRST_VALUE(price) OVER w AS first_price,  
3         LAST_VALUE(price) OVER w AS last_price,  
4         NTH_VALUE(price, 2) OVER w AS second_price  
5 FROM fact_price_change  
6 WINDOW w AS (  
7     PARTITION BY product_key  
8     ORDER BY effective_from, price_change_key  
9     ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING  
10 );
```

9 One frame, shared by all three. Written once, so the three columns cannot disagree about what the partition contains.

THE TAKEAWAY

The named WINDOW clause earns its place here more than anywhere. Three frame-sensitive functions that must agree, defined in exactly one place.

NTH_VALUE, and what is missing

- ◆ **NTH_VALUE takes a position within the frame**, counting from the start. The second sale, the third price, the fifth visit.
- ◆ **The standard defines FROM LAST as well**. Postgres implements only FROM FIRST, and says so in its documentation.
- ◆ **SQL Server has no NTH_VALUE at all**. A filtered ROW_NUMBER is the usual substitute there.
- ◆ **Counting from the end is a reversed ordering**. ORDER BY x DESC with NTH_VALUE n gives the nth from the end, everywhere.

THE TAKEAWAY

Nth from the end is the common request and the least portable syntax. Reversing the ordering is the version that runs on every engine.

First and last, one row per group

```
1 SELECT DISTINCT customer_key,  
2     FIRST_VALUE(product_key) OVER w AS first_ever_bought,  
3     LAST_VALUE(product_key)  OVER w AS most_recent_bought  
4 FROM fact_sales_line  
5 WINDOW w AS (  
6     PARTITION BY customer_key  
7     ORDER BY sold_ts, sale_key  
8     ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING);
```

1 DISTINCT works here only because both windows cover the whole partition, so every row of a customer is identical.

THE TAKEAWAY

A legitimate use of DISTINCT after a window, and a rare one. It collapses rows that are genuinely identical rather than hiding a grain problem.

FIRST_VALUE is not MIN

- ◆ **MIN returns the smallest value in the frame.** FIRST_VALUE returns the value on the earliest row of the frame.
- ◆ **Those agree only when the ordering is the column you are taking.** Order by date and take price, and they are unrelated.
- ◆ **First price is FIRST_VALUE. Lowest price is MIN.** Both are one line, and the wrong one is never obviously wrong.
- ◆ **Scenario two is this mistake**, with MAX standing in for a carry-forward and quietly returning the highest.

THE TAKEAWAY

Earliest and smallest are different ideas that happen to coincide when you sort by the thing you are measuring. Most reports do not.

Where the value functions win

- ◆ **Attribution.** First touch and last touch on a customer journey are `FIRST_VALUE` and `LAST_VALUE` over an ordered partition.
- ◆ **Opening and closing.** Opening balance and closing balance for a period, on every detail row, with no second query.
- ◆ **Baselining.** Every row's value as a ratio to the first row's, which is how an indexed series is built.
- ◆ **All three need the whole-partition frame.** None of them works with the default, and two of them fail silently.

THE TAKEAWAY

First touch works by accident under the default frame and last touch does not. If you only ever test the first, the bug ships with the second.

The previous row might be there and still have **nothing in it**. That is a different question.

IGNORE NULLS

```
1 SELECT product_key, price_date, price,  
2         LAG(price) OVER w          AS prev_row_price,  
3         LAG(price) IGNORE NULLS OVER w AS last_known_price  
4 FROM vw_price_spine  
5 WINDOW w AS (PARTITION BY product_key ORDER BY price_date);
```

- 2 Returns null whenever the previous row happens to be null. Usually most of them.
- 3 Skips nulls entirely and returns the last row that actually had a price. Two words, and a completely different measure.

THE TAKEAWAY

Two columns, two words apart. One answers 'what was on the previous row' and the other answers 'what was the last price we knew'.

IGNORE NULLS, by engine

ENGINE	IGNORE NULLS ON LAG AND LEAD
Oracle	Yes. It has had it the longest.
Amazon Redshift	Yes, on LAG, LEAD, FIRST_VALUE, LAST_VALUE and NTH_VALUE.
Snowflake, BigQuery, Databricks	Yes.
SQL Server 2022 and later	Yes. Added in 2022, with a fix in CU4.
PostgreSQL 18 and earlier	No. The documentation states it is not implemented.
PostgreSQL 19	Yes. Committed for the next major release.

THE TAKEAWAY

The same split as QUALIFY, and in the same direction. Kestrel's Redshift warehouse has this feature and its Postgres 16 replica does not.

Twelve years in a commitfest

- ◆ **The Postgres patch was first submitted in March 2013** by Nicholas White. Tom Lane replied the same day.
- ◆ **It bounced through commitfests for over a decade**, picked up and returned with feedback repeatedly, by several different people.
- ◆ **One contributor argued for it plainly**, calling it among the most needed features in analytics, and noting Oracle and Redshift already had it.
- ◆ **It landed for PostgreSQL 19.** Twelve years from first patch to commit, for a feature that is two words long.

THE TAKEAWAY

A useful calibration. The gap between what the standard defines and what your engine implements can be measured in decades, and it is nobody's fault.

Carry forward without IGNORE NULLS

```
1 SELECT product_key, price_date, price,  
2        FIRST_VALUE(price) OVER (  
3          PARTITION BY product_key, grp  
4          ORDER BY price_date  
5        ) AS last_known_price  
6 FROM (SELECT *,  
7        COUNT(price) OVER (  
8          PARTITION BY product_key ORDER BY price_date  
9          ROWS UNBOUNDED PRECEDING) AS grp  
10      FROM vw_price_spine) t;
```

7 COUNT of a column skips nulls, so it rises only on rows that have a price.

9 Each null inherits the last real price's group number.

THE TAKEAWAY

The portable carry-forward, and the first appearance of the grouping trick module thirteen is built on.

Why that works

- ◆ **COUNT(column) ignores nulls and COUNT(*) does not.** That one difference is the whole mechanism.
- ◆ **A running count over a sparse series produces a step function**, flat across the gaps and rising at each real value.
- ◆ **Rows sharing a step number belong to the same real value**, so grouping on it collects each value with the nulls that follow it.
- ◆ **It needs two query blocks**, because a window cannot partition by another window. Module three's nesting rule.

THE TAKEAWAY

Turning a sequence of nulls into a group number is one of the two or three genuinely general techniques in this course. Module thirteen names it.

Carry forward, or leave the gap

CARRY FORWARD

Assert continuity

- Every day has a price
- Joins and filters behave
- Claims the value persisted
- Hides that nothing was recorded

LEAVE THE NULL

Assert ignorance

- Only real observations appear
- Downstream must handle nulls
- Claims nothing it cannot support
- Makes the sparsity visible

THE TIEBREAKER

A price genuinely does persist until changed, so carrying it forward is true. A sensor reading does not, and carrying it forward invents data.

When not to carry forward

- ◆ **When the absence is the signal.** A member with no events this month has not been quiet, they have lapsed.
- ◆ **When the measure is a flow rather than a stock.** A balance persists. A day's sales do not, and carrying them forward doubles revenue.
- ◆ **When it crosses a boundary that matters,** such as a price carried past the date a product was delisted.
- ◆ **Cap the carry.** Carry forward, but not further than a stated number of days, and null after that.

THE TAKEAWAY

Stock or flow is the question. Carrying a stock forward is restating what was true. Carrying a flow forward is fabricating events.

Kestrel's price-as-at report joins fact_price_change to a date spine, so most days carry a null price and only change days carry a value. On the Redshift warehouse it uses LAG with IGNORE NULLS and is correct.

Ported to the Postgres 16 replica, IGNORE NULLS is rejected. An engineer replaces it with MAX(price) OVER (ORDER BY price_date ROWS UNBOUNDED PRECEDING). The report runs, and margin figures drift upward over time for products whose prices have fallen.

What is the defect?

- A MAX returns the highest price so far, not the most recent one. The portable carry-forward is the running-count grouping.
- B IGNORE NULLS is standard SQL and Postgres supports it. The syntax was simply written in the wrong position.
- C The nulls should be coalesced to zero before the window, so that MAX has no nulls to contend with.
- D LAG needs an offset large enough to reach back to the last price change, which varies by product.

◆ PAUSE AND DECIDE

What is the defect?

- A

 MAX is not latest
A running maximum never falls. For any product whose price has ever dropped, the report holds the old higher price forever.
- B

 Postgres does support it
It does not, in version 16 or 18, and the documentation says so. It was committed for 19, which is not what this replica runs.
- C

 COALESCE the nulls to zero
Makes it worse in the other direction. A running maximum over zeros and prices is still a maximum, and now zero is a legitimate candidate.
- D

 A larger LAG offset
The gap length varies per product and per period, so no constant offset exists. An offset cannot express 'the last one that had a value'.

THE SIGNAL

"Drift upward for products whose prices have fallen." A running maximum is monotonic. A measure that only ever moves one way is the signature of the wrong aggregate.

Almost everything built on LAG is **one of four shapes**. The rest of this module is those four.

Reading a navigation expression

WRITTEN	SAID
<code>LAG(x) OVER (PARTITION BY k ORDER BY t)</code>	x on the row before this one, within k
<code>LAG(x) IGNORE NULLS OVER ...</code>	x on the last row before this one that had an x
<code>LEAD(t) OVER ... - t</code>	how long until the next row, within k
<code>FIRST_VALUE(x) OVER (... whole partition)</code>	x on the earliest row of this k
<code>MIN(x) OVER (PARTITION BY k)</code>	the smallest x anywhere in this k

THE TAKEAWAY

Rows one and two differ by four words spoken and two words written. Rows four and five differ by everything, and look nearly the same.

Percentage change, and its three traps

```
1 SELECT sale_date, daily_sales,  
2        LAG(daily_sales) OVER w AS prev,  
3        (daily_sales - LAG(daily_sales) OVER w)  
4        / NULLIF(LAG(daily_sales) OVER w, 0) AS pct_change  
5 FROM agg_store_day  
6 WINDOW w AS (PARTITION BY store_key ORDER BY sale_date);
```

- 4 NULLIF guards division by zero on a day with no sales. Without it, one closed day fails the whole query.
- 2 The first row is null, so pct_change is null there. That is correct, and the report has to say so rather than show a blank.

THE TAKEAWAY

Three traps in one expression: a null numerator on the first row, a zero denominator on a dead day, and a negative denominator if the measure can go below zero.

Percentage change against zero

- ◆ **Zero to anything is an undefined percentage**, not an infinite one, and not a hundred percent.
- ◆ **NULLIF turns it into a null**, which is honest and which somebody downstream will render as a blank.
- ◆ **Decide what the report should say**, and say it. 'New', 'from zero', or suppressed, are all better than a silent null.
- ◆ **Negative denominators are the subtle one**. A fall from minus ten to minus five is a fifty percent rise, arithmetically.

THE TAKEAWAY

Every one of these is a business decision being made by an arithmetic operator. The only question is whether anybody made it deliberately.

Time between events

```
1 SELECT member_durable_key, event_ts,  
2        LAG(event_ts) OVER w AS prev_event_ts,  
3        event_ts - LAG(event_ts) OVER w AS gap,  
4        CASE WHEN event_ts - LAG(event_ts) OVER w  
5              > INTERVAL '30 minutes'  
6              THEN 1 ELSE 0 END AS new_session  
7 FROM fact_app_event  
8 WINDOW w AS (PARTITION BY member_durable_key ORDER BY event_ts);
```

6 A session boundary flag. The first event of each member has a null gap, so the CASE returns 0 rather than 1.

THE TAKEAWAY

Kestrel's app events carry no session identifier and the business defines a session as thirty minutes of inactivity. This is the first half of building one.

Detecting a change

```
1 SELECT product_key, price_date, price,  
2        LAG(price) OVER w AS prev_price,  
3        CASE WHEN price IS DISTINCT FROM LAG(price) OVER w  
4              THEN 1 ELSE 0 END AS changed  
5 FROM fact_price_change  
6 WINDOW w AS (PARTITION BY product_key ORDER BY price_date);
```

3 IS DISTINCT FROM, not <>. On the first row the previous price is null, and <> would return unknown rather than true.

THE TAKEAWAY

With <>, the first row of every product is silently not counted as a change, and so is any transition into or out of a null price.

Three comparisons, one window

```
1 SELECT retail_week_key, weekly_sales,  
2         LAG(weekly_sales, 1) OVER w AS last_week,  
3         LAG(weekly_sales, 4) OVER w AS four_weeks_ago,  
4         LAG(weekly_sales, 52) OVER w AS last_year  
5 FROM agg_store_week  
6 WINDOW w AS (PARTITION BY store_key ORDER BY retail_week_key);
```

4 52 weeks back on a 4-4-5 calendar. Not 365 days, and not the same date last year.

THE TAKEAWAY

Three offsets, one sort, one window definition. The engine reads a buffer 52 rows deep and computes all three in the same pass.

Year on year, on a retail calendar

- ◆ **The same date last year is the wrong comparison** for a retailer. Last 25 December was a Wednesday and this one is a Thursday.
- ◆ **52 weeks back preserves the day of week**, which is what actually drives grocery trade. That is why 4-4-5 exists.
- ◆ **Except in a 53-week year**, where 52 weeks back is the wrong week and every year-on-year figure shifts by one.
- ◆ **dim_date should carry the comparable week key**, so the answer is a column rather than an offset anybody can get wrong.

THE TAKEAWAY

This is a modelling decision being made inside a window function. The retail calendar exists precisely so that nobody has to make it in SQL.

The comparable period as a column

```
1 SELECT d.retail_week_key, a.weekly_sales,  
2        ly.weekly_sales AS last_year_sales  
3 FROM agg_store_week a  
4 JOIN dim_date d ON d.retail_week_key = a.retail_week_key  
5 LEFT JOIN agg_store_week ly  
6     ON ly.store_key = a.store_key  
7     AND ly.retail_week_key = d.comparable_week_ly;
```

7 The calendar decides what last year means, once, for every report. Not each analyst, in each query.

THE TAKEAWAY

Slower than $\text{LAG}(x, 52)$ and correct in a 53-week year. The comparison rule lives in the dimension, where it can be tested and changed in one place.

A pricing team reports the number of price changes per product per quarter, computed by comparing price with `LAG(price)` using the not-equals operator and counting the rows where it is true.

Finance reconciles against the source system and finds the report is short by exactly one change for almost every product, and short by more for the 1,900 products whose price was temporarily unset during a range review.

What is the cause?

- A LAG needs a default argument of zero, so that the first row compares against a real value rather than a null.
- B The first row of each product should be filtered out, since it genuinely has no previous price to compare against.
- C COUNT DISTINCT on price would avoid the comparison entirely and is the simpler correct answer.
- D Not-equals returns unknown when either side is null, so the first row and every null transition are silently dropped.

◆ PAUSE AND DECIDE

What is the cause?

A Default the LAG to zero

Makes the first row compare true against any non-zero price, which happens to give the right count and asserts that every product started at zero.

C COUNT DISTINCT on price

Counts distinct values rather than transitions. A price that goes 5, 6, 5 is two changes and two distinct values, and the two diverge quickly.

B Filter the first row

Backwards. The first row is one of the missing changes, not a spurious one. Removing it makes the shortfall permanent by design.

D Three-valued logic

Unknown is not true, so those rows are never counted. The null transitions explain why the 1,900 reviewed products are short by more.

THE SIGNAL

"Short by exactly one for almost every product." A uniform off-by-one per group points at the first row of each partition, and the extra shortfall names the nulls.

What LAG cannot do

- ◆ **It cannot skip rows by a condition.** IGNORE NULLS skips nulls and nothing else. There is no IGNORE WHERE.
- ◆ **So make the value null when the condition fails.** LAG of a CASE, with IGNORE NULLS, skips exactly the rows you chose.
- ◆ **It cannot see a value another row computed.** Windows read input columns, never each other's output. Module three.
- ◆ **So a compounding or carried-state calculation is not a LAG.** It is recursion, and it is module eleven.

THE TAKEAWAY

The first two points together are the most useful trick in this module. Turning a predicate into a null is how you make IGNORE NULLS mean anything you like.

The previous promotional sale

```
1 SELECT customer_key, sold_ts, promo_key,  
2         LAG(CASE WHEN promo_key IS NOT NULL  
3             THEN sold_ts END)  
4         IGNORE NULLS OVER w AS prev_promo_ts  
5 FROM fact_sales_line  
6 WINDOW w AS (PARTITION BY customer_key ORDER BY sold_ts, sale_key);
```

- 2 The CASE makes the timestamp null on non-promotional rows, so IGNORE NULLS skips exactly those.
- 5 Every row is still in the output, including the non-promotional ones. Filtering first would have removed them.

THE TAKEAWAY

The predicate has been converted into a null so that an existing keyword can act on it. Every row survives, which is what separates this from a WHERE clause.

A loyalty analyst wants, for every sales line, the timestamp of that customer's previous purchase that was on promotion. The output must keep all lines, promotional or not, because the report also shows what was bought in between.

Their first attempt uses LAG with an offset of 2, reasoning that roughly every other line is promotional. The figures are wrong in a way that varies by customer, and they ask what the correct approach is.

What should they be told?

- ☐ A Use an offset of 3 instead, since 62% of lines are non-promotional rather than half of them.
- ☐ B Apply IGNORE NULLS to LAG of sold_ts, which will skip the non-promotional rows automatically.
- ☐ C LAG cannot skip by predicate. Make the value null when the condition fails, then LAG that with IGNORE NULLS.
- ☐ D Filter to promotional lines before the window, which makes every previous row a promotional one by construction.

◆ PAUSE AND DECIDE

What should they be told?

A Use an offset of 3

Replaces one wrong constant with another. Promotional lines are not evenly spaced, so no fixed offset can find the previous one.

C CASE to null, then IGNORE NULLS

The predicate is converted into a null so an existing keyword can act on it, and every row stays in the output.

B IGNORE NULLS on sold_ts

sold_ts is not null on non-promotional rows, so there is nothing for the keyword to skip. It would change nothing at all.

D Filter first

Gives the right previous timestamp and drops every non-promotional line. The scenario requires those lines, which is what rules it out.

THE SIGNAL

"The output must keep all lines." That single requirement eliminates the filter, and once filtering is off the table, converting the predicate to a null is the only route left.

What navigation costs

- ◆ **LAG and LEAD are cheap once the data is sorted.** A small buffer of rows, carried along, with no accumulation.
- ◆ **Several of them over one specification share a sort,** so five LAGs on the same window cost barely more than one.
- ◆ **IGNORE NULLS is not free.** The engine may scan back an unbounded distance to find a non-null, and Snowflake caps the search.
- ◆ **The sort is the cost.** As it is for every window function, which is module twenty one.

THE TAKEAWAY

Repeating a LAG expression three times in one SELECT is fine. The engine computes it once, and the redundancy is a readability problem rather than a cost one.

Testing a LAG

- ◆ **Two groups in the fixture, not one.** A single group cannot catch a missing PARTITION BY, and that is the commonest defect.
- ◆ **Assert the first row of the second group is null.** That one assertion catches the boundary bleed on its own.
- ◆ **Include a null value in the middle,** so that IGNORE NULLS and RESPECT NULLS produce visibly different answers.
- ◆ **Include a gap in the series,** so previous row and previous period disagree and the test says which you meant.

THE TAKEAWAY

Four rows of fixture, three assertions, and every defect in this module becomes visible. None of them is visible on complete, single-group test data.

The fixture that catches it

```
1 -- store 1: days 1, 2, 3      store 2: days 1, 2
2 -- s1 d1 100      s1 d2 NULL  s1 d3 120
3 -- s2 d1 500      s2 d2 510
4
5 -- with PARTITION BY store_key:
6 --   s2 d1 prev = NULL      <-- the assertion
7 -- without PARTITION BY:
8 --   s2 d1 prev = 120      <-- reads store 1
```

6 Five rows of fixture. The only row that differs between correct and incorrect is the first row of the second group.

THE TAKEAWAY

Five rows, one assertion. A missing PARTITION BY is invisible on every other row of the test and on every row of production until somebody reconciles.

Every partition has two edges

- ◆ **One first row and one last row, always.** A million customers means a million null LAGs and a million null LEADs.
- ◆ **Those nulls are not rare.** On a table with many small partitions they can be most of the rows.
- ◆ **A customer with one purchase is entirely edge.** Previous and next are both null, and there is no middle at all.
- ◆ **So handle the edge as the common case,** not as an exception to tidy up later.

THE TAKEAWAY

Kestrel has 1.2 million members. A single-purchase member has no previous and no next, and there are a great many of them.

Every defect in this module is **a row that was not there**, or a row that was there and empty.

Choosing, in four questions

01

Position or frame

A fixed offset means LAG or LEAD. A place in the frame means the value functions.

02

Row or value

The previous row, or the previous row that had a value? The second needs IGNORE NULLS.

03

Row or period

The previous row, or the previous day? The second needs a completed series first.

04

The edge

What should the first row of each partition show? Decide it, rather than shipping a null.

THE TAKEAWAY

Steps two and three are the same sentence with a different noun, and they are the two questions people ask with identical English words.

The confusions, settled

SYMPTOM	THE CAUSE
Huge swings on a few rows, moving between runs	No PARTITION BY. Each group reads the previous group.
The comparison reaches back weeks	LAG found the previous row. The series has gaps.
Mostly nulls in the previous-value column	The previous row exists and is null. You wanted IGNORE NULLS.
A carried-forward value that only rises	MAX was used as a carry-forward. It is monotonic.
Change counts short by one per group	Not-equals against a null first row returns unknown.
LAST_VALUE returns the current row	Module five. The default frame ends here.

THE TAKEAWAY

Six symptoms, and five of them produce a number that is plausible. Only the middle one looks obviously odd, and only because the magnitude is large.

Five habits

- ◆ **Write the PARTITION BY before the ORDER BY**, and check it matches the series you mean. Most LAG defects are here.
- ◆ **Decide what the first row shows**, every time. Null, a default, or suppressed, but decided.
- ◆ **Use IS DISTINCT FROM for any comparison against a LAG**. The previous value is nullable by construction.
- ◆ **Guard every division with NULLIF**. A percentage change against zero is not an error to discover in production.
- ◆ **Ask 'previous row or previous period' out loud** before writing the window, because the syntax is identical either way.

THE TAKEAWAY

The third one is the cheapest habit in this course. IS DISTINCT FROM is never wrong where not-equals is right, so there is no case for preferring the operator that can fail.

The four shapes and their edges

SHAPE	THE EDGE IT HAS
Difference against previous	First row is null, and null minus anything is null.
Ratio against previous	First row is null, and a zero previous value divides by zero.
Time to next event	Last row of every partition is null, by construction.
Change detection	First row compares against null, so not-equals returns unknown.
Carry forward	Rows before the first real value have nothing to carry.

THE TAKEAWAY

Five shapes, five edges, and every one of them is at the start or the end of a partition.
There is no shape here whose edge is in the middle.

You have the whole window vocabulary now. **Module seven** puts aggregates back inside it.